



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

PokeGetter - Juego de lluvia

Integrantes:

Rodrigo Escudero

Mateo Cueto

Gonzalo Leal

Diego Binimelis

8 de junio de 2026

1. GM-1: ¿Qué experimenta el jugador?

El juego sitúa al usuario en el rol de un entrenador Pokémon que debe recolectar la mayor cantidad posible de pokébolas que caen desde la parte superior de la pantalla. El objetivo es simple pero desafiante: atrapar las bolas beneficiosas, esquivar las peligrosas y sobrevivir el mayor tiempo posible mientras la dificultad se incrementa de forma dinámica.

¿Cómo se controla la aventura? El jugador maneja un personaje Pokémon (representado en el código por la clase **Tarro**) que se desplaza horizontalmente mediante las teclas de dirección izquierda y derecha. La partida se pausa con la tecla ESC, y al finalizar se puede reiniciar pulsando ENTER o volver al menú principal con ESC.

¿Qué tipos de pokébolas existen? Cada pokébola que cae tiene un comportamiento único. La siguiente tabla resume sus efectos:

Tabla 1.1: Tipos de pokébola, sus efectos inmediatos y variación de puntuación.

Tipo	Efecto al capturarla	Puntaje
Normal	Suma puntos.	+100
Rocket (mala)	Resta 1 vida, resta puntos y activa animación de daño.	-75
Rocket Fuerte	Resta 2 vidas, resta puntos y activa daño.	-55
Curativa	Recupera 1 vida y suma puntos reducido.	+75
Veloz	Duplica la velocidad del personaje durante 5 segundos.	+100
Pesada	Reduce a la mitad la velocidad del personaje durante 5 segundos.	+100

¿Cómo cambia la dificultad? El juego no es estático. A medida que el puntaje acumulado supera los 4000 y luego los 10000 puntos, el sistema aumenta automáticamente el nivel (fácil → medio → difícil). Esto modifica parámetros como la frecuencia de caída, la velocidad de cada tipo y la probabilidad de que aparezcan pokébolas peligrosas. El nivel actual se muestra en pantalla, dando al jugador retroalimentación constante.

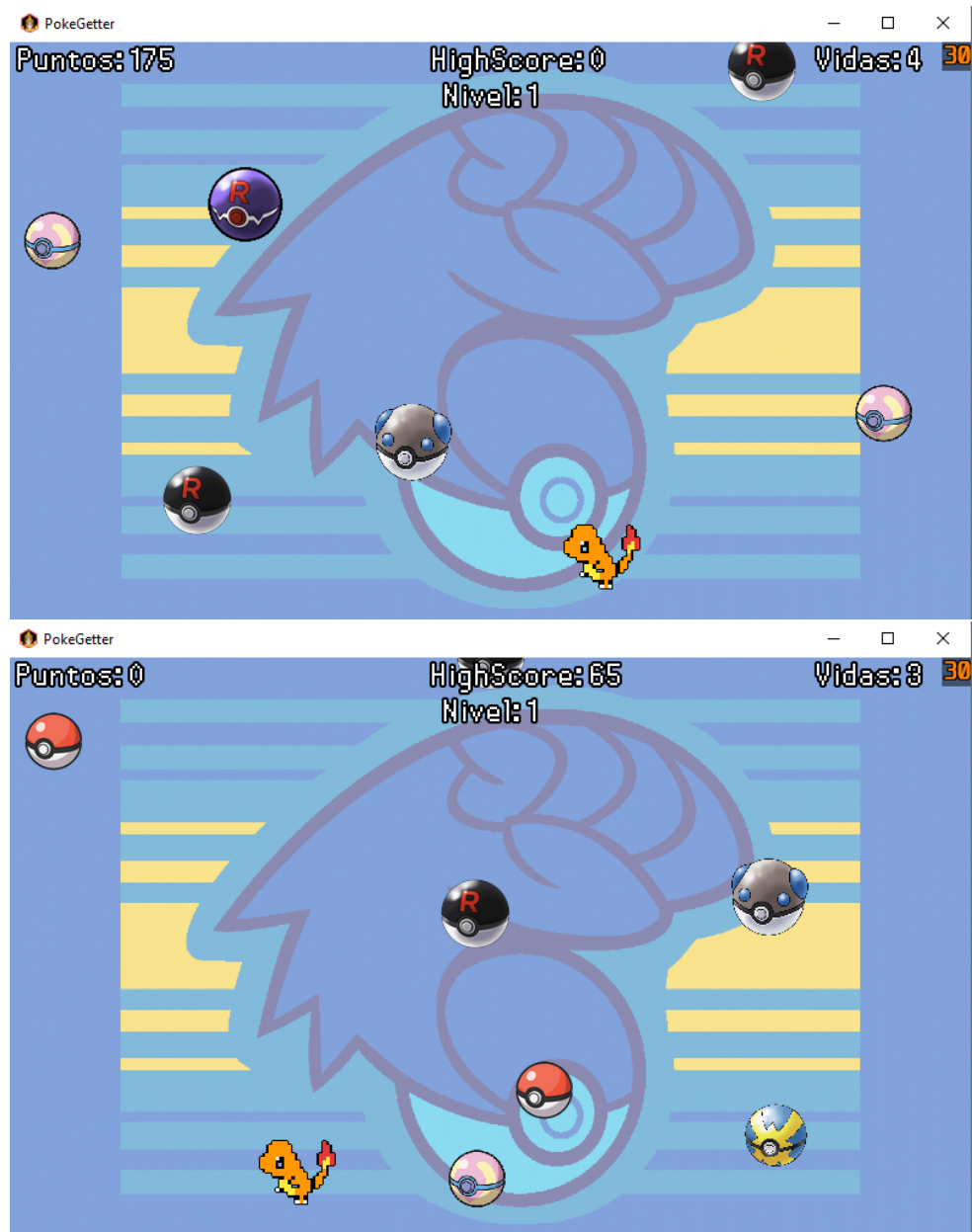


Figura 1.1: Capturas del juego con las 5 pokebolas y los indicadores de vidas, puntaje y nivel.

2. GM-2: ¿Cómo se transformó el código base?

¿De dónde partimos? El proyecto se construyó sobre un tutorial básico de LibGDX que simulaba una lluvia de gotas atrapadas por un personaje. Ese prototipo, aunque funcional, presentaba una arquitectura espagueti: la lógica de juego, la gestión de recursos y el estado del jugador convivían en unas pocas clases sin separación de responsabilidades. Cualquier modificación requería reescribir múltiples partes y la adición de nuevos tipos de objetos resultaba casi imposible sin violar el principio de abierto/cerrado.

¿Qué decisiones de rediseño se tomaron? El equipo aplicó una refactorización completa basada en patrones de diseño y principios SOLID. La siguiente tabla resume las diferencias fundamentales entre el código base original y el sistema implementado:

Tabla 2.1: Comparativa entre el prototipo base y la solución final (GM)

Característica	Prototipo Base (LibGDX)	Sistema Implementado (GM)
Gestión del estado del juego	Variables sueltas dentro de la clase Tarro . Cada vez que se necesitaba el puntaje había que pasar referencias.	Clase EstadoJuego (Singleton) centraliza vidas y puntaje, accesible globalmente.
Carga de texturas	Múltiples <code>new Texture(...)</code> esparcidos en el código. Provocaba duplicación y fugas de memoria.	GestorRecursos (Singleton) carga una sola vez cada textura y las libera controladamente.
Movimiento de objetos	Movimiento lineal rígido: <code>y -= velocidad * delta</code> . No se podía variar sin modificar la clase de la gota.	Patrón Strategy : interfaz EstrategiaMovimiento con implementaciones MovimientoNormal , Rapido , ZigZag . Se asignan en tiempo de ejecución.
Comportamiento de colisión	Bloque <code>if-else</code> gigante dentro de Lluvia que distinguía tipos de gota y aplicaba efectos manualmente.	Patrón Template Method : clase abstracta Pokebola define <code>capturar()</code> como método plantilla. Cada subclase implementa su efecto en <code>aplicarEfecto()</code> .
Configuración de dificultad	Parámetros fijos o repartidos en constantes. Aumentar la dificultad requería cambiar valores en varias líneas.	Patrón Builder : NivelBuilder permite crear niveles de forma fluida y legible. La dificultad cambia automáticamente según el puntaje.
Estructura del proyecto	Código monolítico dentro de unas pocas clases. Los recursos se cargaban directamente en el juego principal.	Separación en paquetes implícita (modelo, vista, controlador). Uso de pantallas (MainMenuScreen , GameScreen , PausaScreen , GameOverScreen).

¿Cómo se organizan las responsabilidades ahora? El sistema sigue una arquitectura dirigida por pantallas, donde cada escena del juego es una clase independiente que implementa la interfaz **Screen** de LibGDX. El flujo de ejecución es el siguiente:

1. **MainMenuScreen**: muestra el mensaje de bienvenida y espera un toque para iniciar la partida.
2. **GameScreen**: contiene el bucle principal de juego. Allí se instancian el personaje (**Tarro**), la **Lluvia** y se gestionan los eventos de entrada, pausa y fin del juego.
3. **PausaScreen**: se superpone al juego cuando el usuario pausa, y al reanudar se restaura la pantalla anterior.
4. **GameOverScreen**: presenta la puntuación final, el récord histórico y ofrece opciones de reinicio o salida al menú.

¿Qué beneficios aporta esta nueva arquitectura?

- **Extensibilidad**: agregar una nueva variedad de pokébola solo requiere crear una subclase de **Pokebola** e implementar **aplicarEfecto()**. No se toca **Lluvia** ni el resto del sistema.
- **Mantenibilidad**: cada clase tiene una responsabilidad clara y acotada. Cambiar la lógica de movimiento de un tipo de pokébola no afecta a los demás.
- **Reutilización**: las estrategias de movimiento pueden combinarse con cualquier pokébola sin duplicar código.
- **Pruebas**: el uso de interfaces y la inyección de dependencias facilita la creación de pruebas unitarias (por ejemplo, se puede probar una estrategia de movimiento sin necesidad de una pantalla completa).

¿Dónde se evidencia esta transformación en el código? Los cambios están documentados en los archivos Java del proyecto. Los nombres de clases y métodos mencionados aparecen explícitamente en el código fuente, y sus relaciones se modelan en el diagrama UML de la sección GM-3. El lector puede verificar la implementación de cada patrón en los archivos **EstadoJuego.java**, **GestorRecursos.java**, **Pokebola.java** y sus subclases, **EstrategiaMovimiento.java** y sus implementaciones, y **NivelBuilder.java** junto con **Nivel.java**.

Figura 2.1: Vista de alto nivel de la arquitectura del juego, mostrando la interacción entre pantallas, el modelo y los patrones implementados.

3. GM-3: ¿Cómo se relacionan las clases del juego?

¿Qué clases de **LibGDX** se utilizan? El proyecto se apoya en la infraestructura que ofrece LibGDX. Las clases base más relevantes son:

- **Game**: clase principal que gestiona las pantallas.
- **Screen**: interfaz implementada por cada escena del juego (menú, juego, pausa, game over).
- **SpriteBatch**: responsable de dibujar texturas en la pantalla.
- **OrthographicCamera**: define la proyección y la vista del mundo.
- **Texture**: representa imágenes (personaje, pokébolas, fondos).
- **Rectangle**: útil para detectar colisiones.
- **Sound** y **Music**: para efectos de audio y música de fondo.

¿Cuáles son las clases propias del dominio? El equipo diseñó una jerarquía de clases que modela fielmente el comportamiento del juego. Las principales son:

- **GameLluviaMenu**: extiende **Game** y actúa como punto de entrada. Contiene el **SpriteBatch**, la fuente global y el récord histórico.
- **MainMenuScreen**, **GameScreen**, **PausaScreen**, **GameOverScreen**: cada una implementa **Screen** y define su propia lógica de renderizado y entrada.
- **Tarro**: entidad controlable por el jugador (el personaje Pokémon). Gestiona su posición, animación de daño y efectos temporales de velocidad.
- **Lluvia**: administra la lista de pokébolas activas. Se encarga de generarlas con diferentes probabilidades y velocidades según el nivel de dificultad, y de verificar colisiones.
- **Pokebola** (clase abstracta): madre de todos los tipos de pokébola. Define atributos comunes (posición, velocidad, textura, límites) y el método plantilla **capturar()**.
- **PokebolaNormal**, **PokebolaRocket**, **PokebolaRocketFuerte**, **PokebolaCurativa**, **VelozBall**, **PesoBall**: subclases concretas que implementan **aplicarEfecto()** según su comportamiento.
- **EstadoJuego** (Singleton): almacena las vidas y el puntaje global. Proporciona métodos para modificar estos valores de forma segura.
- **GestorRecursos** (Singleton): carga y provee todas las texturas del juego. Libera la memoria al finalizar.
- **Nivel** y **NivelBuilder**: permiten construir dinámicamente los parámetros de dificultad (probabilidades, velocidades, tiempo de generación).
- **EstrategiaMovimiento** (interfaz): define el método **mover()**. Sus implementaciones concretas son **MovimientoNormal**, **MovimientoRapido** y **MovimientoZigZag**.

¿Cómo se relacionan todas estas clases? El diagrama UML completo (figura 3.1) muestra las relaciones de herencia, implementación de interfaces, dependencias y el uso de los patrones Singleton, Strategy, Template Method y Builder. A continuación se describen las relaciones más importantes:

1. **Herencia:** **Pokebola** es abstracta y sus seis subclases concretas heredan de ella. **GameLluviaMenu** extiende **Game**. Cada pantalla implementa la interfaz **Screen**.
2. **Implementación de interfaces:** **Pokebola** implementa la interfaz **EfectoCaptura**. **MovimientoNormal**, **MovimientoRapido** y **MovimientoZigZag** implementan **EstrategiaMovimiento**.
3. **Composición:** **Lluvia** contiene una colección de objetos **Pokebola**. **GameScreen** contiene una instancia de **Tarro** y una de **Lluvia**.
4. **Dependencia:** **Lluvia** depende de **EstadoJuego** (Singleton) y de **GestorRecursos** (Singleton). **Tarro** depende de **EstadoJuego** indirectamente a través de las pokébolas que aplican efectos.
5. **Asociación:** **Pokebola** tiene una referencia a **EstrategiaMovimiento** (Strategy). **NivelBuilder** construye objetos **Nivel**.

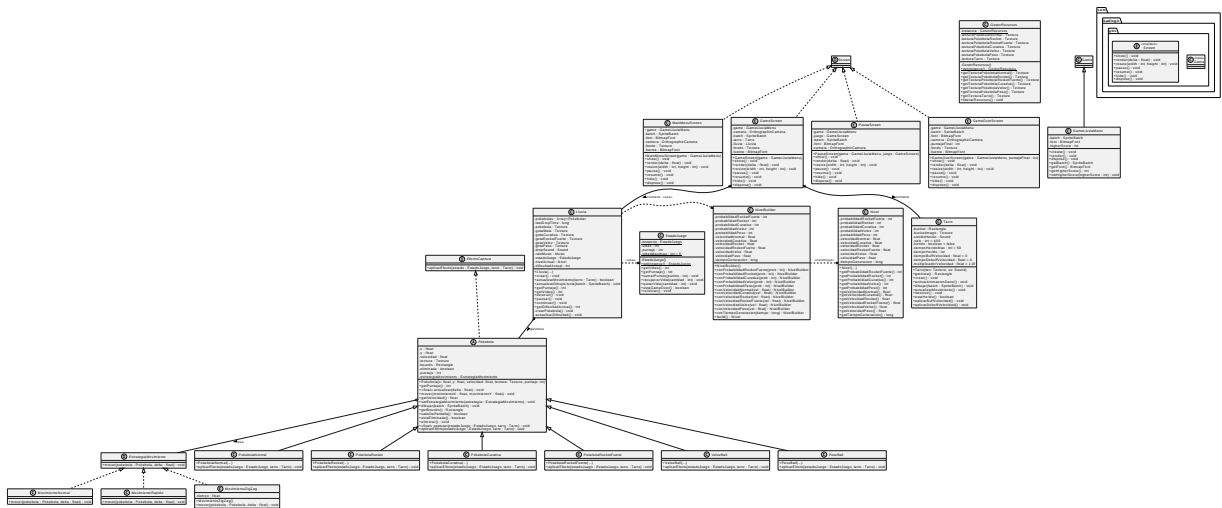


Figura 3.1: Diagrama UML completo del sistema, mostrando todas las clases del dominio, sus atributos, métodos y relaciones. Se destacan los patrones Singleton, Strategy, Template Method y Builder.

4. GM-4: ¿Qué abstracciones definen el comportamiento?

¿Por qué una clase abstracta para las pokébolos? En el dominio del videojuego nunca cae una “pokébola genérica” sin identidad. Cada objeto que desciende debe pertenecer a un tipo concreto (normal, Rocket, curativa, etc.) que determine su efecto al ser capturado. Por esta razón se diseñó la clase **Pokebola** como **abstracta**. Ella encapsula los atributos y métodos comunes a todas las variantes, pero no puede ser instanciada directamente.

La clase abstracta define:

- Atributos privados: posición (`x`, `y`), velocidad, textura, límites de colisión (**Rectangle**) y un indicador de eliminación.
- Métodos concretos: **actualizar()**, **dibujar()**, **mover()**, **getBounds()**, **eliminar()**.
- El método plantilla **capturar()** (declarado `final`), que establece el flujo invariable de captura.
- La delegación del movimiento a una estrategia (**EstrategiaMovimiento**).

De esta forma, cualquier subclase hereda automáticamente toda la mecánica de desplazamiento y colisión, y solo debe preocuparse de implementar su efecto particular.

¿Qué papel cumple la interfaz EstrategiaMovimiento? El movimiento de las pokébolos puede variar drásticamente: unas caen en línea recta, otras se aceleran, otras zigzaguean. Para evitar que la clase **Pokebola** se llene de condicionales, se definió la interfaz **EstrategiaMovimiento**. Ella declara un único método: `mover(Pokebola pokebola, float delta)`.

¿Cuántas clases la implementan? Actualmente existen tres estrategias concretas:

1. **MovimientoNormal**: desplazamiento vertical constante.
2. **MovimientoRapido**: incrementa la velocidad vertical en un factor de 1.35.
3. **MovimientoZigZag**: añade una componente horizontal oscilante (función seno) que simula una trayectoria serpentina.

Gracias a esta interfaz, se puede asignar o cambiar la estrategia de movimiento de cualquier pokébola en tiempo de ejecución mediante `setEstrategiaMovimiento()`, sin modificar la clase **Pokebola** ni las subclases.

¿Cómo se evidencia esta jerarquía en el código? Los archivos del proyecto muestran claramente la relación:

- **Pokebola.java** declara la clase como `public abstract class Pokebola implements EfectoCaptura`. Contiene el atributo `private EstrategiaMovimiento estrategiaMovimiento` y el método `public final void capturar(...)`.
- Las seis subclases concretas (**PokebolaNormal.java**, **PokebolaRocket.java**, etc.) extienden **Pokebola** e implementan `aplicarEfecto()`.
- **EstrategiaMovimiento.java** define la interfaz. **MovimientoNormal.java**, **MovimientoRapido.java** y **MovimientoZigZag.java** la implementan.
- En **Lluvia.java**, dentro del método `crearPokebola()`, se asignan distintas estrategias según el tipo.

¿Qué principios SOLID se respetan aquí?

- **Abierto/Cerrado (OCP)**: la clase **Pokebola** queda cerrada a modificaciones cuando se añade un nuevo movimiento o una nueva variedad de pokébola.
- **Responsabilidad Única**: **Pokebola** gestiona la posición y colisión; cada estrategia se ocupa exclusivamente de un algoritmo de movimiento; cada subclase solo define su efecto.
- **Sustitución de Liskov (LSP)**: cualquier subclase de **Pokebola** puede reemplazar a su padre sin alterar el funcionamiento.
- **Inversión de Dependencias (DIP)**: **Pokebola** depende de la abstracción **EstrategiaMovimiento**, no de implementaciones concretas.

¿Por qué este diseño es mejor que la alternativa con condicionales? Si no se usara una clase abstracta ni interfaces, el código de **Lluvia** tendría que preguntar el tipo de cada pokébola con múltiples `instanceof` para decidir cómo moverla y qué efecto aplicar. Eso generaría un alto acoplamiento y haría imposible añadir nuevas variedades sin modificar decenas de líneas. El diseño actual es limpio, extensible y fácil de probar.

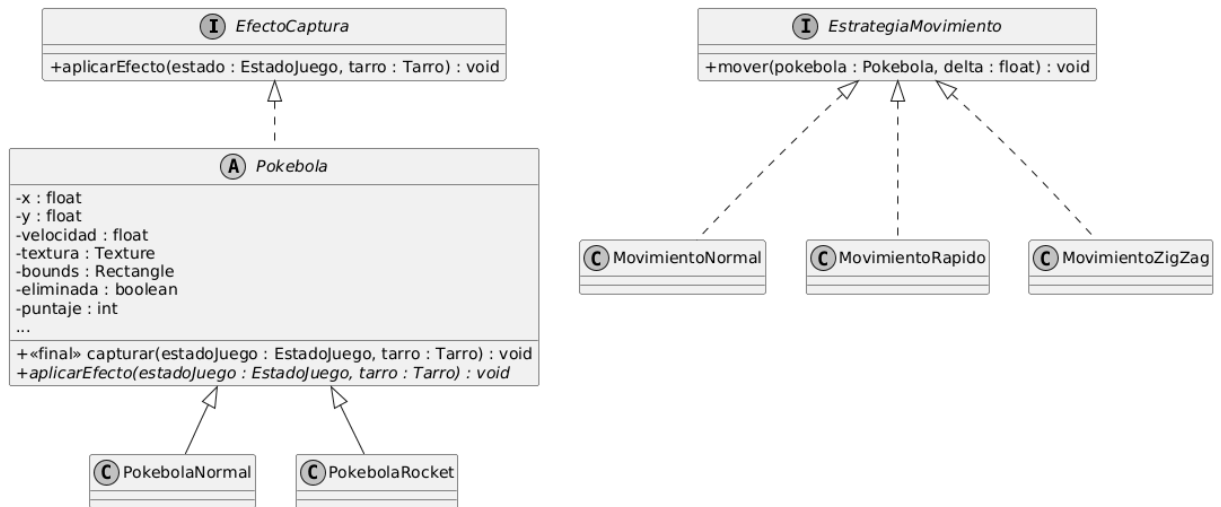


Figura 4.1: Diagrama UML que muestra la clase abstracta **Pokebola** con sus seis subclases, y la interfaz **EstrategiaMovimiento** con sus tres implementaciones.

5. GM-5: ¿Cómo se protegen los datos y se organiza el código?

¿Qué significa encapsulamiento en este proyecto? El encapsulamiento es uno de los pilares de la orientación a objetos. En el juego, todas las clases del modelo declaran sus atributos como **private** o **protected**, y ofrecen métodos públicos solo cuando es estrictamente necesario. Esto evita que otras partes del programa modifiquen el estado interno de forma incorrecta.

Por ejemplo, la clase **Tarro** (el personaje Pokémon) tiene atributos como **tiempoBuffVelocidad**, **multiplicadorVelocidad** y **herido** que son privados. Su modificación solo se realiza a través de métodos públicos como **aplicarBuffVelocidad()**, **aplicarDebuffVelocidad()** o **activarAnimacionDano()**. De esta manera, el comportamiento del personaje queda controlado y cualquier cambio futuro en la implementación interna no afecta al resto del sistema.

¿Qué principios SOLID se aplicaron además del encapsulamiento? El equipo adoptó cinco principios fundamentales (aplicando SRP e ISP "sin querer"):

1. **Responsabilidad Única (SRP)**: Cada clase tiene una razón de cambio bien definida. **Lluvia** solo gestiona la generación y actualización de pokébolas. **Tarro** solo maneja la entrada del jugador y sus efectos. **EstadoJuego** almacena exclusivamente el estado global.
2. **Abierto/Cerrado (OCP)**: Las clases están abiertas para la extensión pero cerradas para la modificación. Para añadir un nuevo tipo de pokébola, se crea una subclase de **Pokebola** sin tocar **Lluvia** ni las otras clases existentes.
3. **Sustitución de Liskov (LSP)**: Todas las subclases de **Pokebola** pueden reemplazar a su clase base sin alterar el correcto funcionamiento del sistema.
4. **Segregación de Interfaces (ISP)**: Las interfaces **EfectoCaptura** y **EstrategiaMovimiento** son pequeñas y específicas. Las clases que las implementan no dependen de métodos que no utilizan.
5. **Inversión de Dependencias (DIP)**: **Pokebola** depende de la abstracción **EstrategiaMovimiento** y no de implementaciones concretas. Asimismo, **Lluvia** depende de la abstracción **Pokebola**.

¿Dónde se evidencian estas buenas prácticas en el código?

- En **Tarro.java**, todos los atributos son privados. Los métodos públicos como **aplicarBuffVelocidad()** y **aplicarDebuffVelocidad()** modifican el estado interno de forma segura.

- En **EstadoJuego.java**, el constructor es privado y los atributos solo se acceden mediante `getInstance()` y métodos de negocio como `sumarPuntos()`, `quitarVida()`. Esto garantiza una única fuente de verdad.
- En **Pokebola.java**, los atributos como `x`, `y`, `velocidad` son privados, y se exponen solo los getters necesarios (`getBounds()`, `getVelocidad()`). El método `mover()` es protegido y solo se invoca desde la jerarquía de herencia.

¿Qué beneficios concretos aporta esta organización? Gracias al encapsulamiento y a los principios SOLID, el sistema es:

- **Extensible:** agregar una nueva pokébola con un efecto inédito requiere solo crear una nueva subclase e implementar `aplicarEfecto()`.
- **Mantenible:** un error en la lógica de movimiento de las pokébolas se corrige dentro de la estrategia correspondiente, sin afectar al resto.
- **Probable:** se pueden escribir pruebas unitarias para cada estrategia de movimiento de forma aislada, inyectando dependencias simuladas.

Figura 5.1: Diagrama que muestra la visibilidad de atributos y métodos en las clases principales, destacando el uso de `private`, `protected` y métodos públicos controlados.

6. GM-6: ¿Por qué centralizar el estado y los recursos? (Singleton)

¿Qué problemas presentaba el código base original? En el prototipo inicial de LibGDX, las variables de estado (vidas y puntaje) estaban mezcladas dentro de la clase **Tarro**. Cada vez que otra parte del programa necesitaba conocer el puntaje o modificarlo, había que pasar referencias del objeto **Tarro** a través de múltiples clases. Esto generaba un alto acoplamiento y dificultaba el mantenimiento. Por otra parte, las texturas se cargaban con `new Texture(...)` en diferentes lugares, provocando duplicación de objetos en memoria y fugas de memoria (*memory leaks*) al no existir un control centralizado de su ciclo de vida.

¿Cómo resuelve el patrón Singleton estos problemas? El patrón Singleton garantiza que una clase tenga una única instancia y proporcione un punto de acceso global a ella. En el proyecto se aplicó en dos clases independientes:

1. **EstadoJuego**: centraliza el estado mutable del juego (vidas, puntaje, vidas máximas). Ofrece métodos como `sumarPuntos()`, `quitarVida()`, `recuperarVida()` y `estaGameOver()`. Su constructor es privado y se accede globalmente mediante `getInstance()`.
2. **GestorRecursos**: centraliza la carga, el acceso y la liberación de todas las texturas del juego (personaje, pokébolas, fondos). También utiliza constructor privado y el método estático `getInstance()`. Las texturas se cargan una única vez al inicio y se liberan controladamente al finalizar.

Ambas clases eliminan la duplicación de recursos y proporcionan una fuente única de verdad para el estado del juego.

¿Dónde se evidencia su implementación en el código?

- En **EstadoJuego.java**, el constructor es privado y la instancia se obtiene con `getInstance()`. Los atributos de vidas y puntaje solo se modifican mediante métodos como `sumarPuntos()` y `quitarVida()`, lo que garantiza que el estado siempre sea consistente.
- En **GestorRecursos.java**, todas las texturas se cargan dentro del constructor privado. El método `getInstance()` retorna la única instancia, y `liberarRecursos()` se encarga de liberar la memoria al cerrar el juego.
- El uso práctico se observa en **GameScreen.java**, donde se obtienen las texturas llamando a `GestorRecursos.getInstance().getTextura...()`. También en **Lluvia.java**, se accede al estado del juego mediante `EstadoJuego.getInstance()`. Finalmente, al cerrar la aplicación, **GameLluviaMenu.java** invoca `liberarRecursos()` para evitar fugas de memoria.

¿Qué principios SOLID se refuerzan con este patrón?

- **Responsabilidad Única (SRP):** **GestorRecursos** se ocupa exclusivamente de la gestión de recursos gráficos, y **EstadoJuego** solo de la lógica de estado. En el código base, ambas responsabilidades estaban dispersas en varias clases.
- **Acceso global controlado:** Se evita el paso repetitivo de parámetros y se reduce el acoplamiento, facilitando el mantenimiento y las pruebas.

¿Qué hubiera pasado si no se usara Singleton? Sin Singleton, cada vez que se necesitara el puntaje se tendría que pasar el objeto **Tarro** como parámetro a través de varias capas (por ejemplo, desde **GameScreen** hasta cada **Pokebola**). Las texturas se cargarían múltiples veces, desperdiciando memoria y provocando lentitud. El Singleton resuelve elegantemente ambos problemas.

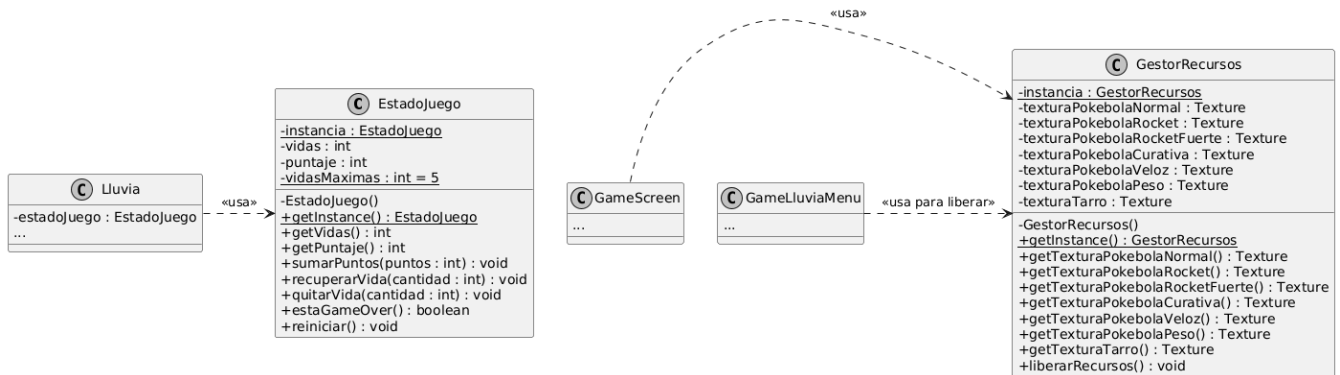


Figura 6.1: Diagrama UML que modela la implementación del patrón Singleton en las clases **EstadoJuego** y **GestorRecursos**.

7. GM-7: ¿Cómo se varía el movimiento sin modificar las clases? (Strategy)

¿Qué limitación tenía el movimiento en el código base? En el prototipo original, todas las gotas caían con un movimiento lineal idéntico: simplemente se decrementaba la coordenada y en cada fotograma mediante una velocidad fija. Esta rigidez impedía tener distintos comportamientos como caídas rápidas, trayectorias en zigzag o movimientos erráticos. Si se quisiera añadir una nueva forma de movimiento, había que modificar la clase de la gota o la lógica de **Lluvia**, violando el principio de Abierto/Cerrado.

¿Cómo resuelve el patrón Strategy esta situación? El patrón Strategy define una familia de algoritmos, los encapsula y los hace intercambiables. En el juego se creó la interfaz **EstrategiaMovimiento** con el método `mover(Pokebola pokebola, float delta)`. A partir de ella se implementaron tres estrategias concretas:

1. **MovimientoNormal**: desplazamiento vertical constante (velocidad base).
2. **MovimientoRapido**: incrementa la velocidad vertical en un factor de 1.35.
3. **MovimientoZigZag**: añade una componente horizontal oscilante (función seno) que produce una trayectoria serpentina.

La clase abstracta **Pokebola** contiene una referencia a **EstrategiaMovimiento** y delega en ella la actualización de la posición dentro de su método `actualizar(float delta)`. De esta forma, se puede asignar o cambiar la estrategia de movimiento de cualquier pokébola en tiempo de ejecución mediante `setEstrategiaMovimiento(...)`, sin necesidad de modificar la clase **Pokebola** ni sus subclases.

¿Dónde se observa esta implementación en el código?

- **EstrategiaMovimiento.java**: define la interfaz con el método `mover`.
- **MovimientoNormal.java**: implementa el desplazamiento vertical puro.
- **MovimientoRapido.java**: usa un factor multiplicador 1.35 en la velocidad.
- **MovimientoZigZag.java**: acumula tiempo y aplica `MathUtils.sin(tiempo * 5f) * 120f * delta` para el movimiento horizontal.
- En **Lluvia.java**, dentro del método `crearPokebola()`, se asignan distintas estrategias según el tipo: `new MovimientoZigZag()` para **PokebolaRocketFuerte**, `new MovimientoRapido()` para **PokebolaRocket**, y `new MovimientoNormal()` para la mayoría.

¿Qué principios SOLID se benefician con este patrón?

- **Abierto/Cerrado (OCP)**: la clase **Pokebola** queda cerrada a modificaciones para añadir nuevos movimientos. Solo se crea una nueva clase que implemente **EstrategiaMovimiento**.
- **Responsabilidad Única (SRP)**: la lógica de movimiento se extrae de **Pokebola**, que se enfoca en posición y colisión. Cada estrategia encapsula un algoritmo de movimiento específico.
- **Inversión de Dependencias (DIP)**: **Pokebola** depende de la abstracción **EstrategiaMovimiento**, no de implementaciones concretas.

¿Cómo se vería el código sin Strategy? Sin este patrón, la clase **Pokebola** tendría un enumerador o una variable de tipo para decidir el movimiento, con una estructura **switch** o **if-else** dentro de **actualizar()**. Cada nuevo movimiento implicaría modificar esa clase, aumentando la probabilidad de errores y violando el principio de Abierto/Cerrado. Con Strategy, el sistema es limpio, extensible y fácil de probar.

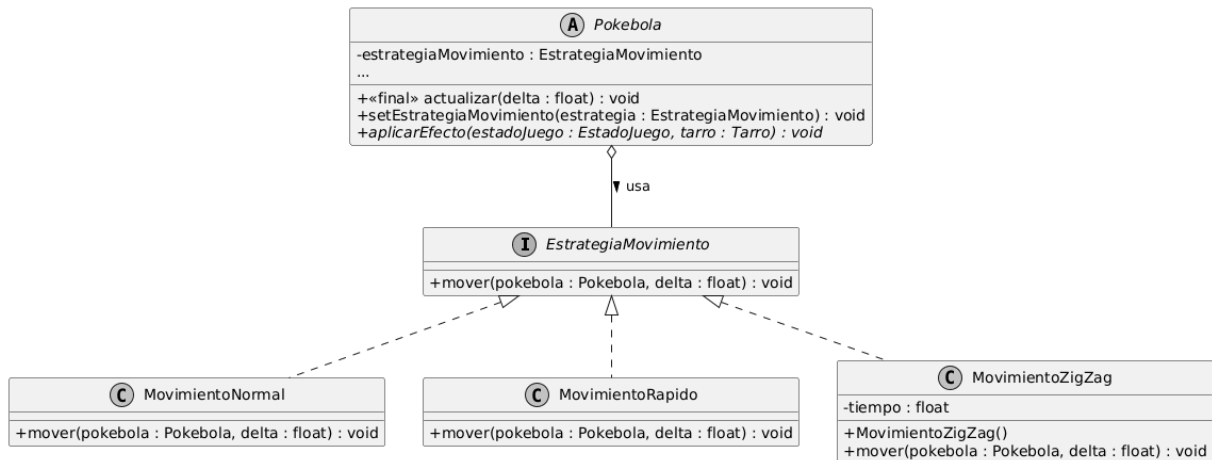


Figura 7.1: Diagrama UML que muestra la interfaz **EstrategiaMovimiento** y sus tres implementaciones concretas, así como la relación de composición con **Pokebola**.

8. GM-8: ¿Cómo se define el esqueleto de la captura? (Template Method)

¿Qué problema presentaba la lógica de colisión en el código base? En el prototipo original, la colisión entre el personaje y las gotas se manejaba con un extenso bloque condicional dentro del bucle principal de **Lluvia**. Cada tipo de gota requería una rama **if-else** que aplicaba el efecto correspondiente (sumar puntos, restar vidas, etc.) y luego eliminaba la gota. Esta aproximación provocaba duplicación de código, acoplamiento entre la generación de objetos y su manejo de colisiones, y una violación del principio de Abierto/Cerrado: añadir un nuevo tipo de pokébola obligaba a modificar la clase **Lluvia**.

¿Cómo resuelve el patrón **Template Method** esta situación? El patrón **Template Method** define el esqueleto de un algoritmo en una clase base, delegando algunos pasos a las subclases mediante métodos abstractos o “gancho”. En el juego, la clase abstracta **Pokebola** implementa el método **capturar()** como **final**. Este método establece una secuencia invariable que se ejecuta cada vez que el personaje atrapa una pokébola:

1. Invoca el método **aplicarEfecto(EstadoJuego, Tarro)**, que es implementado por cada subclase concreta.
2. Invoca el método **eliminar()**, que retira la pokébola de la escena.

De esta forma, el algoritmo de captura permanece siempre igual, mientras que el comportamiento específico (curar, dañar, aplicar buff, etc.) queda encapsulado en cada subclase. El código de **Lluvia** simplemente invoca **pokebola.capturar(estadosJuego, tarro)** sin necesidad de conocer el tipo concreto de pokébola.

¿Dónde se evidencia esta implementación en el código?

- En **Pokebola.java**, el método **capturar()** está declarado como **public final void capturar(EstadoJuego estadoJuego, Tarro tarro)**. Dentro de él se invoca **aplicarEfecto(estadosJuego, tarro)** y luego **eliminar()**.
- **PokebolaNormal.java**: implementa **aplicarEfecto()** sumando el puntaje.
- **PokebolaRocket.java**: resta una vida, resta puntaje y activa la animación de daño.
- **PokebolaCurativa.java**: recupera una vida y suma puntaje reducido.
- **PokebolaRocketFuerte.java**: resta dos vidas y activa daño.
- **VelozBall.java**: aplica **tarro.aplicarBuffVelocidad()**.
- **PesoBall.java**: aplica **tarro.aplicarDebuffVelocidad()**.
- En **Lluvia.java**, dentro del bucle de colisiones, se invoca **pokebola.capturar(estadosJuego, tarro)** sin distinguir el tipo.

¿Qué principios SOLID se favorecen con este patrón?

- **Responsabilidad Única (SRP)**: la clase base define el algoritmo de captura, mientras que cada subclase solo define su efecto particular. La clase **Lluvia** ya no contiene lógica de efectos.
- **Abierto/Cerrado (OCP)**: añadir una nueva variedad de pokébola solo requiere crear una nueva subclase de **Pokebola** e implementar `aplicarEfecto()`. No se modifica **Lluvia** ni la clase base.
- **Sustitución de Liskov (LSP)**: todas las subclases pueden ser tratadas como **Pokebola** y el método plantilla funciona correctamente sin importar la subclase concreta.

¿Qué pasaría si no se usara **Template Method**? Sin este patrón, la clase **Lluvia** tendría que contener una estructura condicional que pregunte el tipo de cada pokébola (`instanceof` o un enumerador) para aplicar el efecto correspondiente. Cada nuevo tipo obligaría a añadir otra rama `if-else`, haciendo el código cada vez más extenso, propenso a errores y difícil de mantener. El **Template Method** encapsula la variación donde corresponde: dentro de cada subclase.

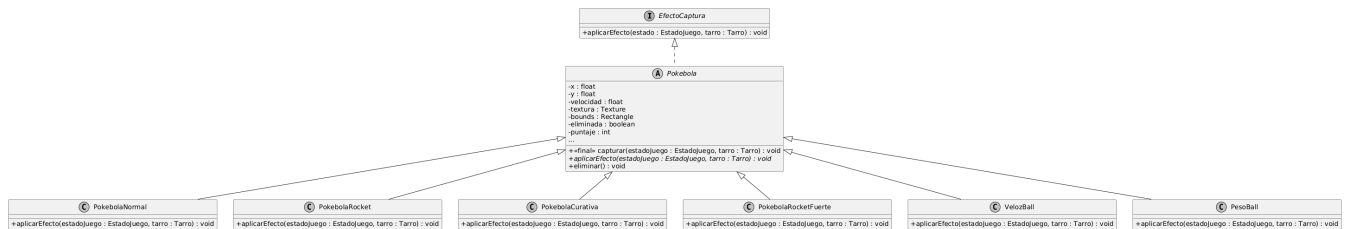


Figura 8.1: Diagrama UML que muestra la clase abstracta **Pokebola** con el método plantilla `capturar()` (final) y el método gancho `aplicarEfecto()`, junto con sus seis subclases concretas.

9. GM-9: ¿Cómo se construyen niveles complejos de forma legible? (Builder)

¿Qué dificultad presentaba la creación de niveles en el código base? En el prototipo original, los parámetros de dificultad (velocidades, probabilidades, frecuencia de caída) eran constantes fijas o estaban dispersas en varias líneas dentro de la clase **Lluvia**. Para escalar el juego y tener distintos niveles (fácil, medio, difícil), se necesitaba configurar múltiples valores: probabilidades de aparición para cada tipo de pokébola, velocidades específicas por tipo, y el tiempo de generación entre objetos.

Una aproximación ingenua habría sido crear un constructor con once parámetros, lo que produce el antipatrón del *Constructor Telescópico*. Este enfoque hace que el código sea difícil de leer, propenso a errores (por ejemplo, intercambiar dos parámetros del mismo tipo), y muy poco mantenible. Además, crear los tres niveles requeriría repetir la misma estructura una y otra vez.

¿Cómo resuelve el patrón Builder este problema? El patrón Builder separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes variantes mediante una interfaz fluida (*fluent interface*). En el proyecto se diseñó la clase **NivelBuilder**, que mantiene valores por defecto (correspondientes al nivel medio) y ofrece métodos encadenables como `conProbabilidadRocketFuerte(int)`, `conVelocidadNormal(float)` y `conTiempoGeneracion(long)`.

El proceso de construcción sigue estos pasos:

1. Se crea una instancia de **NivelBuilder** (con valores predeterminados razonables).
2. Se invocan los métodos `con...()` que se desean modificar. Cada método retorna la propia instancia del builder (`return this`), permitiendo encadenamiento.
3. Finalmente, se llama al método `build()`, que instancia un objeto **Nivel** con todos los parámetros configurados.

El constructor de **Nivel** es de paquete (sin modificador público) y recibe todos los parámetros, pero queda oculto al código cliente, que solo interactúa con el builder.

¿Dónde se evidencia esta implementación en el código?

- **NivelBuilder.java**: declara atributos privados para cada parámetro. El constructor inicializa valores por defecto. Los métodos `con...()` modifican el atributo correspondiente y retornan `this`. El método `build()` invoca al constructor de **Nivel**.
- **Nivel.java**: su constructor recibe once parámetros, pero es invocado únicamente desde `NivelBuilder.build()`. Las clases externas nunca instancian **Nivel** directamente.

- En **Lluvia.java**, los métodos `crearNivelFacil()`, `crearNivelMedio()` y `crearNivelDificil()` utilizan el builder de forma declarativa. Por ejemplo, para el nivel fácil:

```
new NivelBuilder()  
    .conProbabilidadRocketFuerte(5)  
    .conProbabilidadRocket(20)  
    .conProbabilidadCurativa(35)  
    .conVelocidadNormal(260)  
    .conTiempoGeneracion(700000000)  
    .build()
```

- El método `actualizarDificultad()` crea dinámicamente el nivel correspondiente según el puntaje actual del jugador.

¿Qué principios SOLID se refuerzan con este patrón?

- **Responsabilidad Única (SRP)**: la clase **Nivel** solo almacena datos de configuración. La construcción compleja se delega en **NivelBuilder**. **Lluvia** no necesita conocer los detalles de creación.
- **Segregación de Interfaz implícita**: el builder proporciona métodos específicos para cada parámetro, evitando que el cliente tenga que pasar valores que no necesita modificar (los valores por defecto ya están predefinidos).
- **Legibilidad y mantenibilidad**: la creación de niveles mediante encadenamiento de métodos es auto-documentada y mucho menos propensa a errores que un constructor telescópico.

¿Cómo se vería el código sin Builder? Sin este patrón, se necesitaría un constructor como `Nivel(int probRF, int probR, int probC, int probV, int probP, float velN, float velC, float velR, float velRF, float velV, float velP, long tiempo)`. Al invocarlo, el programador tendría que recordar el orden exacto de once parámetros, lo que facilita confusiones. Además, para crear los tres niveles se repetirían casi los mismos valores, generando código redundante. El Builder elimina estos problemas.

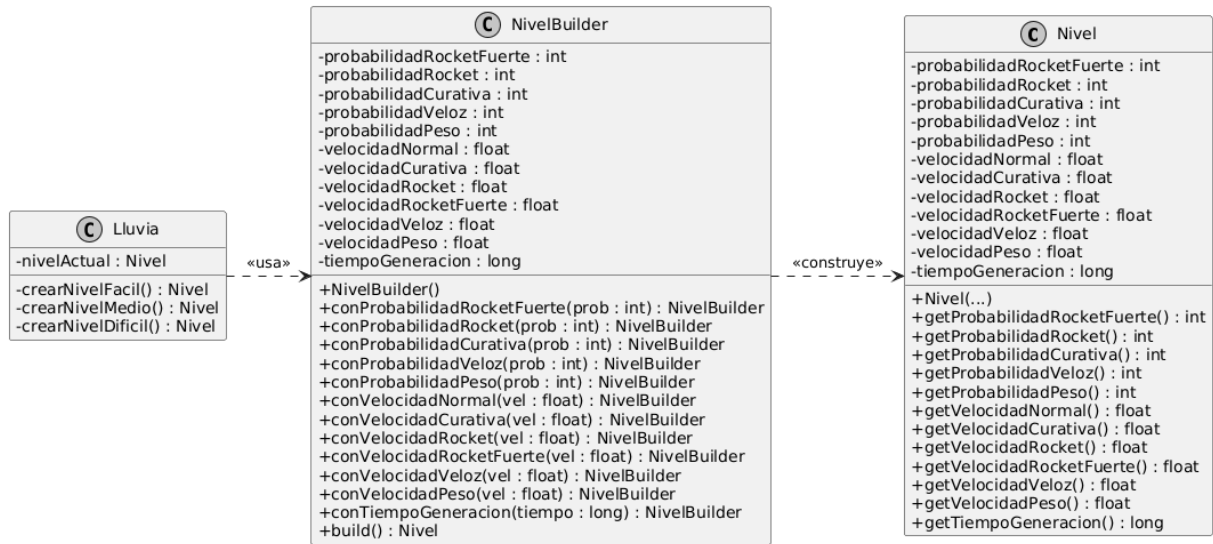


Figura 9.1: Diagrama UML que muestra la clase **NivelBuilder** con sus métodos encadenables, la clase **Nivel** con su constructor de múltiples parámetros, y la relación de uso desde **Lluvia**.

10. GM-10: Utilización de GitHub

El proyecto se gestionó mediante un repositorio en GitHub, accesible en:

<https://github.com/YungRodri/PokeGetter>



Figura 10.1: Commits realizados en Github